FACILITATOR COPY

Facilitation Notes

All 99 slides, with timing, cues and contingencies

Workshop: Agentic AI for Teaching & Academic Processes

Symbiosis Institute of Technology, Pune

Facilitator: Mr. Dinesh Wadhwani

Day 2: 10 September 2026, 1.30 pm – 4.00 pm

Day 3: 11 September 2026, 9.30 am – 4.00 pm

Companion: PPT1_Workshop_Slides.pptx

For the words themselves: the participant text, Parts 1–3

How to use this document

Notes for all 99 slides, in order. The same notes are embedded in the .pptx under each slide.

A NOTE ON WHAT THIS DOCUMENT IS, AND IS NOT

These are **facilitation notes** — what to do, when to pause, what to watch for, what to say if something breaks.

They are not a script. Where you want the actual words for a topic, the participant text is the source: it is written as continuous prose and you can read from it. The mapping is below.

Slides to book chapters

Deck section	Book chapter	Part
1 — What actually changed	Ch 1	1
2 — Prompts that survive contact	Ch 4	1
3 — Your first Actor	Ch 5	2
4 — What is actually happening in there	**Ch 2 and 3**	1
5 — Adding intelligence	Ch 4, 6	2
6 — From sequence to agent	Ch 7	2
7 — Tools	Ch 8	2
8 — Multi-agent	Ch 9	3
9 — The attendance system	Ch 1.6, 11	1, 3
10 — Grounding and guardrails	Ch 10, 11	3
11 — How would you know if it worked?	**Ch 12**	3
12 — From workshop to practice	Ch 13, 14	3

The reading split, and why it matters

Before Day 2: Chapters 1 and 4. **Overnight, before Day 3:** Chapters 2 and 3. **After the workshop:** Parts 2 and 3 in full.

Section 4 assumes they have read Chapters 2 and 3. It is not new material to them — it is you making it solid.

DO NOT PROTECT THE ARITHMETIC SURPRISE

Earlier versions of this workshop revealed the arithmetic failure in Section 7 as a shock.

That no longer works, because Chapter 2 explains it and they will have read it. So Section 7 is now framed as **prediction then confirmation**: "You know why this is about to fail. Predict the answer before I run it."

For an audience that teaches engineering, a predicted failure confirmed is stronger than a magic trick. It is the difference between a demo and an experiment.

Timing

Block	Time
DAY 2 — 1.30 to 4.00 pm	
Opening and contract	10 min
Section 1 — What actually changed	12 min
Section 2 — Prompts	25 min
Lab Tests 1 & 2	65 min
Section 3 — Your first Actor	18 min
Lab Test 3A begins	20 min
DAY 3 — 9.30 am to 4.00 pm	
Recap and framing	8 min
Section 4 — The machinery	20 min
Section 5 — Adding intelligence	12 min
Lab Tests 3A finish, 4A & 5A	75 min
Section 6 — From sequence to agent	20 min
Lab Test 6A	55 min
Lunch	60 min
Section 7 — Tools	20 min
Lab Tests 7A & 7B	75 min
Section 8 — Multi-agent	20 min
Lab Test 8A	50 min
Section 9 — The attendance system	18 min
Section 11 — Evaluation	15 min
Section 12 — Close	12 min

This is still over budget, and deliberately so. Decide what you are dropping before Thursday morning:

Priority	Content	Drop?
Essential	Sections 4, 6, 7. Labs 6A, 7A.	Never
High	Section 11 evaluation, Lab 8A	Only under real pressure
Medium	Lab 9 attendance, Lab 10A, capstone	Shorten before cutting
Droppable	Lab 3B, Lab 7B (Track B)	First to go

Recommended shape: cut Track B labs, run Lab 9 at 60 minutes rather than 90, and give the capstone 60. If Track B was never installed, this happens automatically.

Eight things to repeat

NON-NEGOTIABLES

1. **Personal Gmail, not institutional.** Institutional accounts hide features with no error message.
2. **Add the API key, THEN rebuild.** Environment variables apply at build time. Catches a third of the room.
3. **Never put a key in an input field.** Input values are stored with the run record.
4. **maxIterations = 5**, set before the first run.
5. **Three items, never thirty**, while building.
6. **Memory at 512 MB** before every Apify run.
7. **Export every dataset** — seven-day retention.
8. **Revoke both keys** at the end of Day 3.

Track B only: **JavaScript Code node, never Python.**

The three moments to rehearse

Section 7 — the arithmetic. Take predictions from the room first, out loud. Then run it and write the number on the whiteboard. Then run it twice more and write those beside the first. Three answers to one arithmetic question, and the room predicted it. The true figure is **55.00 per cent, 11 of 20**. Leave that whiteboard up.

Lab 8A — the moderator overrules the grader. The Actor logs `*** MODERATOR OVERRULED THE GRADER: 8 -> 5 ***`. S003 is correct but awkward; S004 is fluent and wrong. Rehearse until the overrule reliably fires.

Section 9 — the three students. R004 falling, R007 recovering, R011 recently contacted. Ask the room what they would do about R007. Someone will say "leave them alone", which is exactly right and which no threshold can express.

If something breaks

Problem	Response
One or two people stuck	Volunteer buddy, move on. Do not hold the room.
Whole room getting 429s	Confirm own keys. Pause 60 seconds, resume.
Nobody can reach the API	Proxy blocking `generativelanguage.googleapis.com`. Escalate; use pre-recorded captures.
`console.apify.com` blocked	Content filter on cloud IDEs. Tier 1 blocker.
Builds queuing	Stagger by row. Half read the loop while half build.
20+ minutes behind	Drop Lab 9 to a demo. Protect Sections 4 and 7.
Running ahead	Extend the capstone. Never extend a lecture section.

One thing to check on Monday

Pin the **exact free-tier Gemini model identifier** and write it on the board. Every Actor takes the model as an input field defaulting to `gemini-2.5-flash`. The free-tier lineup shifted during 2026. One wrong identifier across forty machines is forty HTTP 400 errors.

---

1. Agentic AI for Teaching

& Academic Processes

Open by acknowledging the morning. Dr. Nilima Zade has just spent 2.5 hours on Generative AI, agents and ethics. Do NOT re-teach any of it.

Say something close to: "This morning you saw what an agent IS. This afternoon you build one."

Set the contract immediately — by 4pm today every person in this room will have built something that works. Not seen a demo. Built one.

Watch the room: it is straight after lunch. Get them typing within 15 minutes or you lose them. Do not spend more than four slides before Lab Test 0.

2. The contract for these two sessions

State the deliverable plainly. Faculty have sat through AI sessions before that were all demo and no output; name that and distinguish this one.

The last bullet matters. Say it out loud: "I am going to show you this failing several times today. That is deliberate. If you leave here thinking this technology is reliable, I have done you harm."

Engineering faculty are sceptical by training. Leading with the failures buys credibility that a polished demo never will.

3. The order we build in

This slide justifies the entire structure. Spend a moment on it.

The failure mode of AI workshops is starting at maximum complexity. People nod, feel behind, and build nothing afterwards.

Say: "We are going to make you hit a wall before I hand you the tool that gets over it. That way the tool means something."

Tie it back — this is how they teach. You do not open a DBMS course with query optimisation.

4. What you leave with

Point out that these four are not illustrations — they are a curriculum. Each teaches a genuinely different concept.

Ask for a show of hands: who has an internal assessment to set in the next three weeks? Who has attainment to compute this semester? Usually most of the room. Name that: "Then you are not learning a technology today, you are doing work you already had."

If someone asks why not more use cases: seven shallow demos teach nothing. Four built properly transfer.

5. SECTION 1 — What actually changed

Short section, 10 minutes maximum. It is conceptual and they have had concept all morning.

Your job here is one idea: the difference between a workflow and an agent, and the fact that most of their work needs the former.

6. [statement] When you use a chatbot,

you are the runtime.

This is the framing line for the whole workshop. Deliver it slowly.

Ask: "How many of you have asked ChatGPT or Gemini to generate quiz questions?" Most hands. "How many of you still have that as something you can re-run in ten seconds today?" No hands.

That gap is what the next day and a half is about. Everything we build is an answer to one question: how much of that process moves out of your head and into something that runs on its own?

7. Three tiers — and confusing them wastes months

Walk down the table. Then stop on the last row and read it again.

"Cost to trust" is the row everyone skips and the row that decides whether any of this survives contact with real academic work. An agent is not free once built — it needs supervising forever.

If you only make one point in this section, make it this one.

8. [statement] Do the steps change

depending on what I find?

Give them the question and let it sit for a beat.

Then apply it live. Ask the room: "Computing CO attainment from a marks sheet — do the steps change?" No. Fixed formula. That is a workflow.

"A student asks you a doubt — do the steps change?" Yes, entirely. That is an agent.

They now have a decision rule they can use on Monday without any of the tools.

9. The advice nobody gives you

This is contrarian and they will remember it. Say it directly.

There is a whole industry incentive to make everything an agent. Resist it on their behalf.

Concrete example: someone will build an "attainment agent" that produces a slightly different number each run and cannot work out why. Because they built a creative system for a deterministic task.

Land the last line as the takeaway.

10. Two platforms, deliberately

Apify is the spine. Two reasons, and say both.

Nothing installs — it all runs in a browser tab, and their work sits in their own account rather than on a machine that gets reimaged.

And they write the loop themselves. n8n's Agent node, CrewAI, LangGraph all hide it. Configure one and you have learned an import statement. Write one and every framework afterwards is looking up method names.

Flag now that n8n is Track B and may not run. If the machines were not ready, those labs are dropped and nothing essential is lost. Say this without apology — it was designed that way on purpose.

11. [statement] n8n hides the loop.

The Actor shows it.

This is the framing for every comparison lab today and tomorrow.

They will build the first flow both ways in Lab 3, and the attainment agent both ways in Lab 7. Tell them now that the comparison is the exercise, not a bonus.

If Track B is not running, still say this line — it explains why you chose to have them write code rather than configure a node.

12. SECTION 2 — Prompts that survive contact

Do not let anyone treat this as preamble. Say explicitly: every agent we build tomorrow is a prompt with plumbing around it.

If the prompt is vague, adding tools and memory and multi-agent review produces vague output more elaborately and at greater expense.

This section is about 25 minutes then straight into Lab 1.

13. Where most faculty start

Actually run this on the projector. Do not describe it.

Read one or two of the generated questions out loud, then ask the room to tell you what is wrong with them. They will find it themselves — no CO tags, no marks, all at recall level, one topic they do not teach.

Letting them diagnose it is far more effective than you listing the faults.

14. [statement] Vague prompts do not produce

wrong answers. They produce ave

This reframes what they just saw. The model did not fail — it was told almost nothing and filled every gap with the most typical answer from the whole internet.

Average is worse than wrong for teaching purposes, because wrong gets caught and average gets used.

15. RCTFG — five components

Give them the mnemonic and tell them where the faults usually are.

In practice the missing pieces are almost always F and G. People describe the task well and then leave the output shape and the refusal rules unstated.

Tell them: if a prompt is not working, one of these five is missing. Go through them in order and find which.

16. C — the line that does the most work

Emphasise the NOT TAUGHT line hard. This is the single most transferable thing in the section.

Models are trained on the whole internet's database material and drift toward the mean of it. Telling it what is OUT of scope is far more effective than hoping it stays in scope.

Tell them: every prompt you write for a course should carry this line. It takes thirty seconds and it stops out-of-syllabus questions appearing on your paper.

17. G — guardrails

In Lab 1 they will test this directly: ask for a 5NF question before adding guardrails (it complies) and after (it refuses).

Tell them that comparison is the most valuable thirty seconds in the lab. One paragraph of instructions is the difference between a tool that quietly puts out-of-syllabus questions on their paper and one that does not.

18. Three techniques worth the time

The justification trick is the one to press. It is the highest-value habit in the section and it comes back tomorrow in the attainment agent as a safety feature.

Say: "You are building yourself an audit trail. You will spot a bad tag in one second instead of never."

On Bloom's inflation — give the BCNF example out loud, it lands immediately with engineering faculty.

19. [statement] Iterate on the prompt,

not the output.

Simple, and consistently ignored. The instinct when output is wrong is to fix the output.

Make them promise to fix the prompt instead. This is the difference between using AI and building something with it.

20. What prompting cannot fix

Close the section here and deliver the top line slowly.

"Fluency and correctness are separate properties, and this technology delivers the first far more reliably than the second."

The relocation point is the honest sell. It is still a large saving. It is a different job. Pretending otherwise is how institutions get burned.

Then send them to Lab 1.

21. LAB TEST 1 & 2 — Prompt engineering, then make it permanent

Setup notes before releasing them:

They must be on a PERSONAL Gmail. Institutional accounts hide the Gems option entirely with no error message. Say this twice; it will still catch two people.

Gems are created web-only at gemini.google.com on a computer, not the mobile app.

Walk the room during Lab 1. The common stall is people writing a short prompt and expecting the layered improvement — push them to actually paste the full blocks.

Checkpoint before the break: everyone has generated a question paper for a subject they actually teach.

22. SECTION 3 — Your first Actor

Deliberately no AI in this section. Say why: they need to see data go in, get processed and come out before adding a component whose output varies between runs.

Debugging when you cannot tell whether the plumbing or the model is at fault is unpleasant. Get the plumbing solid first.

23. Every Actor is three sections

This is the whole platform model and it takes ninety seconds to teach.

Emphasise that the agent they build tomorrow has exactly this shape too. It is not a different kind of thing — the middle section is just longer.

Removes a lot of the intimidation of a new platform.

24. [statement] Put your prompts in the input schema.

The single most important design decision on this platform, and the thing that makes it bearable to work on.

Builds take one to two minutes. If prompts are hardcoded, prompt iteration — which is most of what they will do — runs at two minutes per attempt. Unusable.

If prompts are inputs, they can try six versions in two minutes.

Give them the question to ask of every value in their code: am I going to want to change this without changing the logic? If yes, it goes in the schema.

25. Two things that cost you money or time

Both are set on the Input tab, not in code, so there is no excuse for forgetting.

On retention: faculty have lost workshop output this way, intending to use it the following month. Say it as a real thing that happens, not a footnote.

26. The Apify cost trap — two minutes that saves your $5

Show the arithmetic. A location Actor at roughly $0.0040 per result means $5 covers about 1,250 results, and then runs stop.

This is why every lab has them building their own tools rather than calling Store Actors.

Memory is set on the Input tab, not in code, so there is no excuse for forgetting it. Say it here and repeat it before every lab.

27. [statement] Data in n8n is a list.

Every node runs once per item.

Only needed if Track B is running.

This is the idea that unlocks n8n and causes ninety per cent of beginner confusion. Draw it if you can.

Then make the contrast explicit — it is the same lesson as the agent loop, one level down.

28. LAB TEST 3A & 3B — Your first Actor — then the same thing in n8n

Insist on the hand verification. It is the habit that catches everything later.

Common stalls in 3A: JSON syntax error in input_schema.json breaking the build; forgetting to set memory before Start.

The step that matters is changing pass mark from 40 to 70 and re-running with no rebuild. Make sure everyone does it — that is where the fast-path lesson lands.

If Track B is not running, skip 3B and give the comparison verbally.

29. Agents that use tools,

check each other, and run on your own material

Recap in one line: "Yesterday's Gem answers. Today's agent acts."

The difference is three things — it can call a TOOL, it can LOOP, and a second agent can CHECK it. Everything today is one of those three.

Then go straight to the setup gate. Nobody moves to Lab 4 until every machine has printed a response from the API. Two people will still be stuck — assign a buddy and move on. Do not hold 38 people for 2.

30. SECTION 4 — What is actually happening in there

Twenty minutes, and it is the highest-value twenty minutes of Day 3.

They read Chapters 2 and 3 overnight. This is not new material to them — it is you making sure it is solid, and setting up everything that follows.

Say why it matters up front: "By the end of this section you will be able to predict how this technology fails. That is worth more than knowing what it can do."

31. [statement] A language model is a function

that returns a probability di

Deliver this slowly and let it sit.

Then name what is absent, because that list is the whole section: there is no database. No stored facts. No arithmetic unit. No module that decides whether it knows something.

There is a very large function, and a loop.

For an audience that teaches computing, this is a genuinely interesting claim rather than a disappointing one. Treat it that way.

32. The prediction loop, in full

Walk the loop once, then draw out the three consequences on the next slide.

Worth pausing on line 4: sample, not argmax. That single word is why the same prompt gives different answers, and you will come back to it.

33. Three consequences, immediately

The third bullet is the one to press, because it is the foundation of the whole failure story.

Ask the room: "How does the model know the difference between a question it can answer and one it cannot?"

Let them work it out. The answer is that it does not, and there is nowhere in the architecture for that knowledge to live.

34. Numbers tokenise by text frequency, not place value

This slide does a lot of work and takes ninety seconds.

Column-wise addition — the algorithm they were all taught — requires access to individual digits aligned by place value. Tokenisation splits by how often a fragment appears in text, which has nothing to do with arithmetic structure.

So the model is not doing addition badly. It has nothing to do addition WITH.

This is the first of three reasons arithmetic fails, and the most concrete.

35. Temperature, and what to set it to

Temperature flattens the distribution before sampling. Low means near-deterministic; high means low-probability tokens get a realistic chance.

The important caveat, and say it explicitly: low temperature reduces variation, it does not fix correctness. A low-temperature model samples consistently from the same distribution. If that distribution favours a wrong answer, you get the wrong answer consistently.

Consistency and correctness are different properties. People conflate them constantly.

36. Where the knowledge lives, and three regimes

Take these one at a time. The third is the one they will meet.

Then land the point: nothing in the architecture distinguishes these three cases. It is the same operation on the same function. The probabilities are just the probabilities.

Their doubt agent, ungrounded, will invent a worked example in their own notation at the right difficulty level, and no student could tell.

37. [statement] It has no mechanism for

representing the difference between

This is the sentence to write on the whiteboard, if you write one thing.

It reframes hallucination from a defect awaiting a patch into a structural property. That matters because it changes what they do about it: you cannot prompt your way out of an architectural limit, you design around it.

Everything in Sections 7, 10 and 11 is designing around this slide.

38. Why arithmetic fails — three separate mechanisms

This is the slide that makes Section 7 land.

Say plainly: there are far too many possible sums of sixty two-digit numbers for any of them to have been memorised. The model must generalise, and what it generalises is the FORM of an answer, not the value.

Then the five-second diagnostic, which they should use forever: run it three times. If the number moves, it was never computed.

39. The shape of the failure is what makes it dangerous

Let the room go quiet here.

Engineering faculty already half-suspect LLMs are bad at arithmetic. You are confirming a suspicion and then, in Section 7, handing them the fix. That is a much stronger position than surprising them with a failure.

The eight-months line is the one that lands. Say it slowly.

40. [statement] It is fluent before it is correct.

The sentence to carry through both days.

There is no grammar module and a separate truth module. One mechanism produces both, which means the polish of the output tells you nothing about whether it is right.

Flag forward to Section 8: this is also why a grading model rewards fluent writing over correct content. Same mechanism, different consequence.

41. Symptom to cause to fix — keep this

Tell them this table is in Chapter 3 and worth keeping somewhere findable.

The reason it matters professionally: when a colleague brings them a failure in November, the useful answer is not "AI is unreliable." It is "that is confident invention on absent material, here is the line that prevents it."

That is the difference between having attended a workshop and having understood something.

42. SECTION 5 — Adding intelligence

Short and practical, twelve minutes. This is plumbing, and they need it before Lab 4A.

Everything here is a consequence of Section 4. Say so as you go — the rate limits are the only thing that is not.

43. What changes when you add an AI call

The second row is the whole slide. Read it aloud.

"A broken deterministic Actor throws an error and stops. A broken AI Actor completes successfully and hands you something that looks correct."

Every design decision for the rest of today follows from that sentence. Come back to it when you introduce the moderator this afternoon.

44. Two things about the API key

The rebuild gotcha will catch about a third of the room. Say it twice and it will still catch some.

On the input-field point: this is a real security habit, not pedantry. Exported code and run records travel.

Remind them again at the end of Day 3 to revoke both keys.

45. Rate limits — this will affect you today

Spend a full two minutes here. It prevents the most common failure of the day.

Nothing today runs calls in parallel. Everything is sequential with a delay. Slow and complete beats fast and half-failed.

If someone hits the daily quota, they are done for the day on that key — pair them with a neighbour rather than letting them keep clicking.

46. The parser that prevents the most failed runs

Models wrap JSON in code fences habitually despite being told not to. Fighting it is a waste of time; handle it.

Three lines of defence, and tell them to put it in every parser they ever write.

Also flag the try/catch around each call: without it, one malformed response kills the whole run and they lose everyone else's results.

47. LAB TEST 4A & 5A — An Actor that calls Gemini, then structured output

The prompt-iteration step is the one to push. Make them do four iterations on the Input tab and watch the clock — two minutes for four attempts. That is only possible because the prompt is an input.

In Lab 5A there is a deliberate trap: Bhavesh scores 42 per cent, the rule says "strictly below 40". Some models still set needs_meeting to true — applying judgement over the stated rule. Make sure people notice it. That is the silent-error class the whole afternoon is about.

Also have them deliberately break the parser and watch the run survive.

48. SECTION 6 — From sequence to agent

Open with the wall. Show them a question a flow cannot answer, then hand them the thing that can.

49. [statement] "I did not understand the 3NF example

from Tuesday's class.

Run this against a plain flow first so they see the generic answer.

Then decompose why it fails: steps 1 and 5 are the problem. In a flow you decide in advance what happens at every step. Here, what happens depends entirely on what the student said and how they respond.

This is the boundary. Everything after this slide is about crossing it.

50. Three additions turn a flow into an agent

Emphasise the parenthetical on Tools: you do not tell it which to use. That delegation of choice is the definition.

On the loop — this is what costs money and time. Three to ten API calls instead of one. Flag that now, because the next slide is about the bill.

51. [statement] An agent is a model choosing

its own path, one step at a tim

The definition to leave on screen while they build.

If anyone asks about ReAct, planner-executor or the agent taxonomies from yesterday morning — those are names for variations on this loop, and they will see the loop itself in code in ten minutes. Do not go down the taxonomy route.

52. The entire loop, in five steps

This slide is the payoff of choosing to write the loop rather than configure a node.

Say plainly: "Everything in agent engineering is refinement on those five lines of structure."

Then tell them what they get from having written it — they can change the stopping condition, add a validation step, log whatever they want. None of that is possible inside a node you cannot open.

53. A tool is a description and a function

Three rules for a tool description: what it does, exactly what it needs, and when to use it.

"Does calculations" is useless. The one on screen is usable.

Point out that the ONLY thing that changes between the agents they build today is this tools object. The sixty lines of loop stay identical. That is the lesson.

54. Set maxIterations to 5. Before your first run.

Make them set it in the lab as step one, not step nine.

The second instruction is less obvious and just as important — over-cautious looping is more common than runaway looping in practice.

In Lab 6A they deliberately set it to 1 and watch the cap fire. Make sure they do that.

55. [statement] Give the agent fewer tools

than you think it needs.

Counterintuitive and true. The instinct is to give it every capability available.

"Every tool you add makes every decision harder."

Add a third only when they have watched it genuinely fail for want of it.

56. LAB TEST 6A — Your first agent loop — written by hand

Bullet one is not optional. Give them four silent minutes to read the loop and find the five parts before anyone clicks Build.

Then press the trace point hard. It is the only way to understand WHY an agent did something. An agent that gave a wrong answer usually shows exactly where it went astray.

Debugging by re-reading output is guesswork. Debugging by reading the trace is diagnosis.

The negative test in bullet four is the one people skip. If the tool fires when it should not, their description is too broad — and they fix it on the Input tab, no rebuild.

57. SECTION 7 — Tools — where it comes together

This is the payoff of Section 4, and the framing has changed from previous versions of this workshop.

Do NOT present the arithmetic failure as a surprise. They read Chapter 2 and you taught the mechanism this morning. A reveal would fall flat and, worse, would look like you were trying to trick them.

Instead: "You know why this is about to fail. Predict the answer before I run it."

A predicted failure confirmed is stronger than a magic trick, especially for engineers. It is the difference between a demo and an experiment.

58. Predict it before I run it

Run this live. Take predictions from the room FIRST — two or three people out loud.

Then run it and write the number on the whiteboard. Then run it twice more and write those beside the first.

Three different answers to one arithmetic question, and the room predicted it. That whiteboard is the most persuasive object in the two days. Leave it up for the rest of the session.

Most runs land between 60 and 75 — enough to move the attainment level by two full bands, in the generous direction, on a number destined for an accreditation file.

59. Why this matters more in academic work

This is where the room goes quiet. Let it.

Engineering faculty already suspect LLMs are bad at arithmetic — you are confirming a suspicion and then handing them the fix, which is a much better position than surprising them.

The eight-months line is the one that lands. Say it slowly.

60. [statement] The model decides.

The tool computes.

The principle, generalised. Anywhere a number, a date, a lookup or a fact must be exactly right, it comes from a tool.

This single principle separates an agent you can put in front of a department from a demonstration.

Note what did NOT change: same model, same data. Only the division of labour changed.

61. The instruction that does the work

Tell them: an agent willing to do EASY arithmetic itself will also be willing to do hard arithmetic itself, and the hard ones are where it gets things wrong.

So the prohibition has to be absolute, including simple percentages. No carve-outs.

In Lab 7 they will test whether it obeys. If it computes a percentage itself, the instruction needs strengthening.

62. [statement] Make the agent state its inputs

before its result.

The failure a tool does NOT prevent. Make this explicit.

A tool guarantees the calculation is correct. It does not guarantee the agent passed the right arguments. It could correctly call calculate_attainment while passing q2 and q5. The maths would be flawless and the answer wrong.

Same trick as the CO justifications yesterday. Point that back — it is a pattern, not a one-off.

63. LAB TEST 7A & 7B — The attainment agent — built twice

The most important lab. Enforce Part A — people will want to skip to the working version.

The CO7 test in the last bullet is the one that matters most. If it invents a number for a non-existent CO, that is a fabricated figure in an accreditation document. Make everyone run that test and confirm the refusal.

Float continuously during this lab. The common technical stall is the tool input parsing — the query variable arrives as a string and needs JSON.parse.

64. SECTION 8 — Multi-agent

Second-longest section. The payoff moment is the moderator overruling the grader — rehearse until that reliably happens on your demo data.

65. Two failures a single grading agent always commits

This is the ethical core of the day, and it is placed here deliberately — attached to a live consequential decision rather than floating on a slide at 9 am.

The sample data is built for this: S003 is correct but awkward, S004 is fluent and wrong. A single grader typically scores S004 at or above S003.

Have them find it themselves in the lab rather than telling them the answer.

66. The critic's instructions — note the last line

Criteria 3 and 4 exist specifically to catch the bias from the previous slide. Point that out — it is a designed countermeasure, not a generic checklist.

The final paragraph is the load-bearing one. If their moderator rubber-stamps in the lab, this is what they add.

67. What multi-agent costs you

Make them do this arithmetic against their actual class size before deploying anything.

A flow that works on five and dies on eighty fails at exactly the moment they needed it.

Then the honest counterpoint: most of the time multi-agent is not worth it. If a task has one right answer and a deterministic check exists, use the check. Verifying a paper totals 20 marks needs a line of JavaScript, not a reviewing agent.

Use a second agent only for judgements that genuinely require judgement.

68. Before this touches a real mark

Deliver this as practical advice, not a legal disclaimer. It is the difference between a pilot that survives and one that gets shut down after a complaint.

The two quoted positions at the end are the takeaway. Suggest they write the first one into any proposal they put to their department.

If someone says their institution has no policy — that means they are setting precedent. Set it deliberately and in writing.

69. LAB TEST 8A — Multi-agent rubric grading

The comparison in bullet five is the highest-value exercise available to them. It teaches exactly how far to trust this, on material with a ground truth they produced.

If someone's moderator agrees with everything, give them the strengthening line from the manual: "You must find at least one point of disagreement."

Quota warning: three agents across five submissions is about 15 calls per run. One full run is about a minute of quota. Tell them not to run it repeatedly out of curiosity.

70. SECTION 9 — The attendance system

This came from a faculty member in this room, which is worth saying — it is the most directly useful thing in the two days.

It is also where the workflow-versus-agent distinction stops being theoretical, because you build it both ways on the same data and watch the difference.

71. What was actually asked for

Credit the person who asked, by name if they are comfortable with it.

Then be honest about the two problems: it is not an agent as described, and the scheduling will not work the way they expect on a lab machine.

Both are worth ten minutes because both generalise.

72. This is a workflow, and that is not a criticism

Deliver this respectfully but do not soften it. If forty faculty leave believing "scheduled plus branching equals agent", it quietly undoes Section 7.

Then immediately say the workflow version is the RIGHT answer to what was asked, and they will deploy it on Monday. This is not a rejection of the request.

The next slide is where it gets interesting.

73. Three students, one number, three right answers

This table is the whole section. Give the room a moment to read it.

All three sit at roughly the same percentage today. A threshold rule treats them identically, and it MUST, because percentage is the only thing it looks at.

Ask the room what they would do about R007. Someone will say "nothing, leave them alone." That is exactly right, and no threshold can express it.

You could add a rule about the previous month. Then one about days since contact. Then one about repeat cases. Twenty rules later you no longer understand their interactions, which is the fate of every sufficiently elaborated rule engine.

74. [statement] Nobody wrote the rule

about recovering students.

This is the moment the distinction becomes real for them.

Part B gets three tools: current attendance, student history, days-since-a-date. Then a goal rather than a procedure: decide who to contact, who to escalate, who to leave alone, and record a reason.

It looks at the figures, decides it needs history to tell these students apart, goes and gets it, checks how recent the last contact was, and reaches three different conclusions with three different reasons.

Not the schedule. Not the branching. The fact that deciding what to look up is itself part of the task.

75. Drafts by default. Sending is a switch, and it ships off.

Be firm here and brief. This is architecture, not caution.

Two mechanisms from production mail systems that they should adopt permanently:

A recipient override that redirects every message to one address regardless of the data, with the intended recipient in the subject line. They still send real mail and read it in a real inbox, and nothing can escape.

And two conditions to open the gate — override cleared AND a separate acknowledgement. A single checkbox gets ticked absent-mindedly.

76. Four versions, and where each actually runs

The honest recommendation, and say it out loud: for THIS task, Apps Script is the best tool, even though it is neither of the platforms we have been teaching.

It lives in the sheet, reads it natively, sends from their own Gmail, has built-in time triggers, needs nothing installed and no app password, and the student data never leaves Google.

Teaching "here is when not to use the thing I just taught you" is the most credible thing you will do in two days. Do not skip it.

Use Apify when the deciding is genuinely hard — Part B. Use Apps Script when the job is read a sheet and send mail, which is most weeks.

77. LAB TEST 9 — The attendance alert system — built four ways

Bullet two is the one to protect if you run short. Thirty seconds, and it makes the case for review mode better than any warning from you — the drafts start rounding figures and inventing claims about exam eligibility.

R015 in the sample data has a deliberately blank attendance figure. It checks that their code skips the row rather than reading blank as zero and telling a parent their child attended nothing. Make sure someone notices.

Every address in the sample data ends in .invalid, which by internet standard can never resolve. Nothing can reach a real person. Say that once so nobody worries.

78. SECTION 10 — Grounding and guardrails

Flag the stakes at the top. This is the one build where getting it wrong affects someone other than the faculty member.

79. Two ways to ground an agent

Save them three days. Reaching for the vector pipeline first is the most common way to spend a week building something a five-minute approach handles.

Only go further when grounding across a whole programme rather than one course.

80. The three instructions that make grounding work

Explain three properly. Their notation, their simplifying assumptions, their conventions may differ from textbook consensus.

The agent must follow THEM, and flag the difference rather than silently correcting them in front of a student.

Concrete example from the sample material: the course uses 3NF as the practical target and a specific FD notation. A generic model would drift to the textbook version.

81. The Socratic constraint is part of the build

State plainly why this is not a refinement to add later: colleagues will reject the tool outright, and correctly, if it hands out solutions.

The lab tests it under pressure — "I don't have time, just give me the full answer." A constraint that collapses under mild pressure is not a constraint. Make sure everyone runs that test.

Also flag the escalation path: grade queries, deadlines, and any mention of personal difficulty go to a human immediately. Design that FIRST, not last.

82. [statement] Log every question students ask.

Faculty who deploy these consistently report this as the most valuable output.

The answers help students. The QUESTIONS tell them how to teach the course better next semester.

This is a genuinely good closing note for the section — it reframes the agent as a teaching instrument rather than a labour-saving device.

83. LAB TEST 10A — The doubt-solving agent — then test every guardrail

The two capitalised tests are not optional and you should say so.

A Socratic constraint that collapses under mild pressure is not a constraint, and "just give me the answer" is the first thing any student tries.

An agent that attempts to counsel a student in difficulty is a serious problem, not a feature. Correct behaviour is a warm, prompt handoff to a person — not a refusal that reads as a rebuff, and not an attempt to help. Verify it every time the prompt changes.

Also check the opposite failure: if it refuses to explain an unassessed conceptual question, the constraint is too broad and will frustrate students who are learning legitimately.

84. SECTION 11 — How would you know if it worked?

Fifteen minutes, and it is probably what determines whether any of this is still in use in November.

Open by asking the room directly: "You have built a grading agent. It produces scores. The scores look reasonable. Is it any good?"

Let the silence sit. Most people have no answer, and that is the point.

85. What evaluation is not

Tie the first bullet straight back to Section 4. This is the same point as "it is fluent before it is correct", applied to their own judgement of the system.

The last bullet is worth landing carefully. Everyone in the room has a story about the time it produced something brilliant. That story is why they are here, and it tells them almost nothing about reliability.

86. Build a ground truth set — you already have the raw material

Step two is the one people get wrong and it invalidates everything. Emphasise it.

Faculty are unusually well placed for this compared to most people trying to evaluate AI systems — they have thousands of graded submissions sitting in a drawer. Point that out; it makes the task feel achievable rather than academic.

87. Four things to measure

Direction of error matters more than the rate, and it is the one people never look at. A single accuracy figure hides it entirely.

The consistency measure takes five minutes and is the one most likely to disqualify a system outright — two students with identical work receiving different marks is indefensible regardless of average accuracy.

The fourth bullet also has a hidden benefit worth mentioning: sometimes evaluating a system reveals that your RUBRIC was ambiguous, and you had been resolving that ambiguity unconsciously for years. That is a useful finding with or without AI.

88. [statement] Would two of your colleagues

agree with each other more than

This is the frame that makes evaluation tractable rather than paralysing.

The standard is not perfection. Nobody achieves perfection, including them. The standard is whether it falls within the spread that already exists between competent human graders in their department.

That is a question they can actually answer, and answering it usually takes an afternoon.

89. Two things to do, and one to expect

The last point is unusual in software and worth dwelling on for a moment, because it violates an assumption every engineer holds.

They are used to pinning a version. Here they cannot. The thing underneath their prompt can change without any change on their side, and the only detection mechanism is a dated test set they re-run.

Then close with the honest note: be willing to conclude it does not work. If checking the output costs more than doing the work — which happens more than people admit — that is a legitimate finding, and it is more than most people manage.

90. The capstone — pick one lane

Starter code for all five lanes is in the shared folder. Nobody starts from a blank file — say that explicitly, because a blank editor at 1.45 pm is how capstones fail.

Two cores: the agent loop for lanes A, B, D and E, and the pipeline for lane C. Core 1 is byte-identical to Lab 6A, which they have already read line by line.

Only the tools block changes. That is the fourth time today they will have seen that, and by now it should feel obvious.

91. Ten minutes on paper before anyone opens an editor

Enforce the paper stage. Most capstones that fail, fail because nobody decided what they were building.

The clause in bullet three is the useful filter. "Repetitive" is not enough on its own — it has to be something where they would notice a bad answer, because otherwise they cannot supervise it and should not deploy it.

Hand out paper if you need to. Ten minutes of silence.

92. LAB TEST 11 — Capstone — build for your own subject

Bullet one prevents the commonest failure. Paste, build, run, see output, THEN customise. Faculty who customise first debug two things at once and finish with nothing to demo.

The out-of-scope test is worth 20 of the 100 assessment marks. Ask it about data that does not exist — a student, a CO, a topic. It must refuse, not produce a plausible figure. An agent that invents when data is missing is worse than no agent.

The hard stop is not negotiable. Every year somebody demos an untested build and it fails live.

And insist on part three of the demo. A reel of successes teaches nothing and quietly signals that failures are embarrassing. The failures are the transferable content.

93. SECTION 12 — From workshop to practice

Fifteen minutes. Not a futurism slide — a commitment device.

94. Why most of this will not survive

Be blunt. They have all been to workshops that produced nothing, and naming it is more useful than pretending this one is different by default.

Then give them the specific remedy on the next slide.

95. This week — pick one

Row three is the highest-value exercise available to them and worth calling out.

Grade five themselves, run the agent, compare. They learn exactly where it agrees, where it diverges, and how far to trust it — on their own material with a ground truth they produced.

Weeks 1-2: one workflow, one course, used in earnest, note every failure. Weeks 3-4: fix what failed, show one colleague. Their questions find the assumptions you cannot see.

96. If this is to go beyond you

The last bullet is a concrete, actionable thing for whoever runs the department. Mention it specifically to the HoD if present.

On ownership — resist the committee instinct. Name one person or it dies.

Cost honesty: free tier is genuinely sufficient for one teacher on their own courses. Student-facing at cohort scale needs a paid tier. Say so rather than overselling.

97. [statement] The durable skill is not any tool

in this workshop.

Close on this rather than on a forecast. Forecasts age badly and they will remember this line.

Then the final ask on the next slide.

98. One thing, by Friday

Actually make them write it. Pause, hand out paper if needed, wait thirty seconds in silence. The pause is the point.

Then the housekeeping — the key revocation is not optional on shared machines. Walk them through it if there is time.

Propose a follow-up in four to six weeks where people bring what actually worked and what quietly died. The second category is more useful than the first.

Thank the lab team by name. They did two days of unglamorous work to make this possible.

99. Now go and build

something small.

Leave the last slide up during questions.

Point them at the shared folder one more time — study material, lab manual, starter workflows, sample data.

Remind them one final time about key revocation before they walk out.